

Likelihood, Log-Likelihood, and Maximum Likelihood

Why machine learning uses log-likelihood

CSE 534

Goal: compare the probability of data with the plausibility of model parameters

Probability vs. likelihood

Probability: parameters are known; ask what data may appear.

Likelihood: data are known; ask which parameters make the data most plausible.

Example: coin with $\mu = P(\text{heads})$

- ▶ **Probability:** if $\mu = 0.70$, what fraction of heads might we see?
- ▶ **Likelihood:** after seeing 70 heads in 100 flips, which μ is most plausible?

Independent observations

For i.i.d. observations $x_1, \dots, x_n$:

$$L(\theta; x_1, \dots, x_n) = \prod_{i=1}^n p(x_i \mid \theta)$$

Bernoulli example:

$$L(\mu) = \mu^k (1 - \mu)^{n-k}$$

where k is the number of 1s and n is the total number of observations.

The underflow problem

Goal: maximize likelihood.

But multiplying many probabilities drives the product toward zero:

$$L(\mu = 0.7) = 0.7^{70000} \cdot 0.3^{30000} \rightarrow 0$$

Outcome: all candidates underflow to zero, so we cannot compare them.

Demo A: `05a_likelihood_underflow.py`

Log-likelihood solves it

Key idea: products become sums under a logarithm.

$$\ell(\theta) = \log L(\theta) = \sum_{i=1}^n \log p(x_i \mid \theta)$$

Why this helps:

- ▶ maximizing L is equivalent to maximizing $\log L$
- ▶ addition is numerically stable
- ▶ gradients remain usable for optimization

For Bernoulli data:

$$\ell(\mu) = \sum_{i=1}^n [x_i \log \mu + (1 - x_i) \log(1 - \mu)]$$

Maximum likelihood estimate

We want the value of μ that maximizes $\ell(\mu)$.

For Bernoulli observations, the maximum-likelihood estimate is the sample mean:

$$\hat{\mu}_{ML} = \frac{1}{n} \sum_{i=1}^n x_i$$

This is exactly the observed fraction of 1s.

For 70 heads in 100 flips:

$$\hat{\mu}_{ML} = 70/100 = 0.70$$

The derivation

Start from:

$$\ell(\mu) = \sum_{i=1}^n [x_i \log \mu + (1 - x_i) \log(1 - \mu)]$$

Differentiate with respect to μ :

$$\frac{\partial \ell}{\partial \mu} = \sum_{i=1}^n \left[\frac{x_i}{\mu} - \frac{1 - x_i}{1 - \mu} \right]$$

Set it to zero and solve:

$$\hat{\mu}_{ML} = \frac{1}{n} \sum_{i=1}^n x_i$$

Gradient vanishing

Modern ML uses gradient-based optimization.

$$\theta \leftarrow \theta + \alpha \cdot \nabla L(\theta)$$

When raw likelihood becomes extremely small, the gradient collapses:

- ▶ $\nabla L(\theta) \rightarrow 0$
- ▶ parameters stop updating
- ▶ optimization fails

Demo B: `05b_likelihood_gradient_fail.py`

Log-likelihood succeeds

The same idea works if we optimize the log-likelihood instead:

$$\frac{\partial \ell}{\partial \mu} = \sum_{i=1}^n \left[\frac{x_i}{\mu} - \frac{1 - x_i}{1 - \mu} \right]$$

Demo C: `05c_likelihood_logit.py`

This converges to $\mu \approx 0.70$.

Negative log-likelihood

Machine learning usually minimizes a loss function instead of maximizing a likelihood.

$$\text{Loss} = -\ell(\theta) = -\log L(\theta)$$

Key idea:

- ▶ maximizing likelihood $\equiv$ minimizing negative log-likelihood
- ▶ classification uses cross-entropy loss
- ▶ regression often uses Gaussian NLL / MSE equivalence

Why this matters for deep learning

Large language models are trained by minimizing the negative log-likelihood of the observed text:

$$\text{minimize } -\log P(\text{text} \mid \theta)$$

This is the reason the log term appears everywhere in modern ML:

- ▶ more stable numerical behavior
- ▶ better gradients
- ▶ direct connection to model likelihood

Takeaway

Likelihood asks: given parameters, how likely was the observed data?

Log-likelihood asks: which parameters make the observed data most likely, without underflow?

Maximum-likelihood estimation chooses the parameter values that maximize the observed data probability.

Deep learning trains by minimizing negative log-likelihood, which is the same optimization problem in a numerically stable form.